

Branch Testing Cheat Sheet

Oct 8, 2026 · @Syed Hisham Ali

The method in one minute

Branch coverage means every `if / else` outcome and every loop exit runs at least once. You don't need to be clever. You need a checklist and a pen.

1. **List** every branch outcome, one row each, with an ID (C1, C2, H1...).
2. **Pick one row** you haven't covered yet and choose inputs that should reach it.
3. **Trace** the code on paper in a small table, node by node, until you know exactly what happens.
4. **Get the expected value from the spec**, never from running the code. If the trace and the spec disagree, you found a mistake or a bug.
5. **Write a tiny test**: one setup, one call, one or two asserts.
6. **Count your budget, push, read the CI**, then go back to step 2 for whatever is still missing.

The one rule: expected values come from the spec and your own paper working. If you copy what the code returns, a bug in the code becomes a "correct" test.

If you feel lost: you are probably on step 2. Pick the smallest unticked row and do only that one.

Step 1: List the branches

Go through the code top to bottom, one method at a time. Give each method a letter (C for constructor, H for hasNext, N for next, and so on). Every time the code can go two or more ways, write one row per way.

What counts as a branch

In the code	Rows you write
<code>if (x)</code> with no <code>else</code>	2: x true, x false
<code>if / else</code>	2: the <code>if</code> arm, the <code>else</code> arm

In the code	Rows you write
<code>if / else if / else</code>	3: one per arm
<code>a && b</code>	a true, a false, b true, b false (b only runs when a is true)
<code>a b</code>	a true, a false, b true, b false (b only runs when a is false)
Early <code>return</code> guard	2: guard fires, guard doesn't
<code>instanceof</code> check	2: it is that type, it isn't
Loop with <code>break</code>	2: keeps looping, breaks. The <code>if</code> right before the <code>break</code> is the branch.
<code>while (cond)</code>	2: enters/continues, exits
<code>x ? a : b</code>	2
<code>switch</code>	1 per case, plus default

A branch inside a branch gets its own row. For example, an `if (!backwards) push(...)` inside the first arm of an `if / else if / else` has two rows of its own.

Copy this table and fill it in

ID	Branch outcome	How to trigger	Covered by	Done
C1				
C2				
M1				

Tips

- Start the "how to trigger" column with a few words, like "empty tree" or "count = 0". You'll refine it when you trace.
- If a row looks impossible, check the logic before you give up on it. In our iterator, one row looked impossible until we checked which sub-condition could fire.
- Rows that need a special shape of input (like a node whose child has a child) are the ones to plan first. They decide what input you build.

Step 2: Trace it on paper

Pick one unticked row. Choose inputs you think will reach it. Now walk the code by hand, one line at a time, in a table. This is the skill that makes everything else easy.

The trace table

Step	Where am I	Condition checked	True or false	Which branch ran	What changed
1					
2					

Keep going until the code returns or the loop breaks. Then write down the end state (for us it was the stack, top first).

Then get the expected value, separately

1. Ignore the code. Read the spec (or the task description).
2. Work out the right answer in plain words, using paper. For sorted data: write the sorted list, find where the input lands, and read along.
3. Compare with your trace. They should match.

They match	They don't match
Write the test with that expected value.	Stop. Either you traced wrong, or the code has a bug. Re-trace first. If the trace is right, that is the bug you are hunting for.

Worked mini-example (AVLiterator constructor)

Tree: banana, apple on the left, cherry on the right. Call `getRange("blueberry", 5, false)`, meaning start at "blueberry", go forwards.

Node	Node vs begin	Branch that ran	Push?	Move to
banana	smaller	else arm	no (going forwards)	right
cherry	bigger	first arm	yes	left (empty), so stop

Stack, top first: cherry. Spec says "start at the first value at or after begin", and that is cherry. Match, so the expected value is "cherry".

When the input doesn't exist in your data, that is often exactly what you want. Missing values are how you reach the "skip" branches.

Steps 3 and 4: Reuse your input, then write the test

Reuse one input. Don't build a new setup for every branch. Build one small input and change the knobs (start value, direction, count, and so on). Only build a second, different input when a branch needs a shape the first one can't give you. For us: a 3-node tree covered most rows, and one 5-node tree covered the loop rows.

Keep each test tiny. One setup, one call to the thing you're testing, and only the asserts that tick a new box. Extra asserts cost budget and add nothing to coverage.

Skeleton to copy (JUnit4)

```
@Test(timeout=100)
public void saysWhatShouldHappen() {
    // 1. build the input
    AVLTestBuilder<String> source = new AVLTestBuilder<String>
(Comparator.naturalOrder());
    source.setTreeRoot(source.make("banana", source.make("apple"),
source.make("cherry")));
    AVLTree<String> tree = source.getTree();

    // 2. make the call that reaches your target branch
    Iterator<String> iterator = tree.getRange("blueberry", 5, false);

    // 3. assert against your paper answer
    assertEquals("Should skip banana and start at cherry", "cherry",
iterator.next());
}
```

Which assert for what

You want to check	Use	Notes
A yes/no result	<code>assertTrue(msg, x)</code> / <code>assertFalse(msg, x)</code>	Put the real call inside, like <code>iterator.hasNext()</code> . <code>assertFalse("x", false)</code> checks nothing.

You want to check	Use	Notes
A value	<code>assertEquals(msg, expected, actual)</code>	Order is message, expected (your paper answer), actual (the call).
An exception	<code>@Test(expected = SomeException.class)</code>	Only if the spec says it should throw.

Test hygiene

- Name the test after what should happen, not the branch ID.
- Add `timeout=100` if the example test has it.
- Write the branch IDs it covers in a comment above it. It helps you tick the checklist later.
- Build bigger inputs by nesting: `make(value, left, right)`. Use `empty()` for a missing child.

Steps 5 and 6: Budget, push, fix what's missing

Count your budget as you go. For the iterator task the limit was 20 calls total. A call was the constructor (`getRange` counts once), each `hasNext()`, and each `next()`. Building trees and writing asserts cost nothing.

Test	<code>getRange</code>	<code>hasNext</code>	<code>next</code>	Total
Test 1				
Test 2				
Running total				

If you're over budget

- Cut asserts that don't tick a new box (second `next()`, extra `hasNext()`).
- Never cut the only call that exercises a method. We once commented out a `next()` and lost a whole branch.
- Let one test cover several rows. A single `next()` can tick a few boxes at once.

Push and read the CI output

```
git add app/test/YourTests.java
git commit -m "Describe what you changed"
git push
```

If the branch tracks the course repo, use `git push -u origin your-branch` once so it goes to your own fork.

The CI prints three things for the white-box task:

1. **Calls used.** Must be within the limit.
2. **Branch coverage %.** You need 100% for full marks. Ours went 70%, then 88%, then 100%.
3. **Moderated score.** Follows the coverage.

If the % is under 100

1. Tick your checklist honestly against the file CI actually ran. Rows you think are covered may have been commented out.
2. Look for rows with no `next()` or `hasNext()` call exercising them.
3. In IntelliJ, right-click the test class and choose Run with Coverage (JaCoCo runner). The uncovered branches show up highlighted in the source.
4. Fix one row, push, and read the percentage again.

Git gotchas we hit

- `index.lock: File exists` means a stale lock. Make sure no git process is running, then `rm .git/index.lock`.
- Pushing to the wrong remote. `origin` is your fork, `upstream` is the course repo.

Mistakes we actually made

All of these happened in the practice run. Skim them before you start tomorrow.

Mistake	What happened	Fix
Wrong expected value	For <code>begin = null</code> going forwards we first wrote "banana" (the root). The correct first value is the smallest one.	Write the sorted list on paper first. Don't assume the root is where things start.

Mistake	What happened	Fix
Assert on a constant	<code>assertFalse("empty", false)</code> always passes and checked nothing.	Put the real call inside the assert.
Commented out the only <code>next()</code>	Coverage dropped to 70% because whole branches never ran.	Trim extra asserts, never the only call that reaches a branch.
Half-written method	A stray <code>public void</code> with no body stops the whole file compiling.	Delete or finish it before you push.
Left the example test in	It quietly cost 3 of the 20 calls.	Delete starter tests you don't need.
Wrong <code>assertEquals</code> order	Message, expected, actual. Easy to swap.	Say it out loud: message, my paper answer, the call.
Thought black-box and white-box were one score	88% was only the white-box file. The black-box file scored separately.	Read the CI output per task.
Stale <code>index.lock</code>	Git refused to commit.	Check no git process is running, then <code>rm .git/index.lock</code> .
Integrity statement left unfilled or inaccurate	Placeholders still in the file, and the AI answer didn't mention what was actually used.	Fill every question, say what you really used, keep the file's structure.

Rule of thumb: if a test can't fail, it isn't a test. Before you push, ask for each test, "what would make this go red?"

How it played out on AVLiterator

The same loop, run three times. Each push showed what was still missing.

Push	Calls used	Branches covered	What was wrong or added
1	16	70%	Several tests only called <code>hasNext()</code> . The empty-tree test checked nothing. No backwards <code>next()</code> at all.
2	19	88%	Added the 5-node tree tests (backwards refill, and <code>count 0</code>). Still missing: <code>hasNext()</code> on the empty tree, and a backwards pop with no left child.
3	20	100%	Put <code>hasNext()</code> inside the empty-tree assert. Swapped <code>hasNext()</code> for <code>next()</code> in the "avocado" test.

The final set of tests and what each one ticked

Test	Input	Rows it ticked
Empty tree	<code>getRange("apple", 5, false)</code> then <code>hasNext()</code>	Empty root, stack empty
Start at an existing value	<code>begin "banana", forwards</code>	<code>begin</code> equals node, forwards pop with a right child
Start before everything	<code>begin "aaa", forwards</code>	node bigger than <code>begin</code> , forwards push
Missing value, backwards	<code>begin "avocado", backwards</code>	node bigger than <code>begin</code> but backwards (no push), node smaller (push), backwards pop with no left child
Missing value, forwards	<code>begin "blueberry", forwards</code>	node smaller than <code>begin</code> , forwards (no push), forwards pop with no right child
Null <code>begin</code> , forwards	<code>begin null</code>	the null-and-forwards part of the first condition, <code>hasNext()</code> true

Test	Input	Rows it ticked
Null begin, backwards	begin null, backwards	the null-and-backwards route into the else arm
5-node tree, backwards	begin "d", backwards	backwards refill, rightmost loop continues and stops
5-node tree, count 0	begin "d", count 0, forwards	hasNext() false on count 0, next() skipping the decrement, leftmost loop continues

The pattern to reuse tomorrow: one list of branches, one small input, one trace per row, one tiny test per target. Don't try to do it all at once. Do the smallest unticked row, push, and read the number.